Pipelined 8-bit Vector Multiply–Accumulate Architecture

Hubo Zhu

https://github.com/0xCapy/Vector-multiplier

Abstract This report describes a configurable Verilog vector multiply–accumulate (MAC) datapath built from unsigned 8-bit Wallace-tree multipliers, a pipelined adder tree and a shared accumulator. Four elaboration-time configurations provide 1, 4, 8 or 16 parallel multiplication lanes. The implementation separates lane-level multiplication from vector reduction and accumulation, with valid signals propagated through the registered datapath. Module-level and integrated testbenches exercise arithmetic corner cases and streaming inputs. Historical FPGA implementation results for 1-, 4- and 8-lane instances are examined in terms of clock frequency, logic-area efficiency and estimated power. The assessment distinguishes properties visible in the released RTL from results that would require a new simulation or implementation run.

1. Design scope and specification

The datapath evaluates a vector dot product

$$D = \sum_{i=0}^{K-1} a_i b_i, \qquad a_i, b_i \in \{0, \ldots, 255\}, \tag{1}$$

by processing up to N element pairs per accepted input beat. The top-level module, `vector_mac_top_param`, selects $N \in \{1, 4, 8, 16\}$ through the `ACTIVE_LANES` parameter. This choice is resolved at elaboration; the number of instantiated multipliers does not change at run time. The configured vector length defaults to $K = 1000$.

Table 1. Principal RTL interfaces and configuration parameters.

Parameter or interface	Implemented behavior
Operand representation	Unsigned 8-bit values, two operand vectors per beat
Parallelism	$N = 1, 4, 8, 16$ multiplication lanes; 128-bit input buses
Product	16-bit unsigned output from each 8×8 multiplier
Lane reduction	Registered binary adder tree, configured per lane count
Vector accumulation	32-bit running sum, beat count derived from K and N
Control	`vec_valid` input and `result_valid` output; no ready/backpressure port

2. Datapath organization

Each active lane creates one 16-bit product. An adder tree reduces the N products to one beat sum; a 32-bit accumulator adds successive beat sums until the configured vector length has been processed. The public RTL uses separate wrappers for each supported lane count. Additional alignment and output registers are present in some wrappers, so end-to-end latency is configuration dependent.

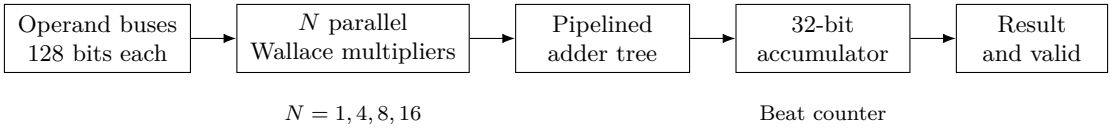


Figure 1. Functional dataflow of the vector MAC datapath; control registers are omitted for clarity.

3. RTL microarchitecture

3.1 Wallace-tree multiplier

The `wallace_mult8` module generates eight shifted partial-product rows using bitwise AND operations, and reduces them through half-adder and full-adder compression logic. Registers divide the reduction network into three pipeline stages. The final carry-propagate operation produces a 16-bit unsigned result. The internal reduction uses a guard bit, but the multiplier's output port is 16 bits, sufficient for the maximum unsigned product.

The RTL attaches a `(* use_dsp = "no" *)` synthesis attribute to the multiplier. This specifies the intended logic implementation; actual DSP utilization must be verified in a synthesis report. The pipeline has an initiation interval of one cycle at the multiplier interface when successive inputs are marked valid.

3.2 Parallel reduction and register alignment

The lane wrappers instantiate N multiplier copies and connect their products to `adder_tree_var`. The tree contains registered binary addition levels; its depth increases with lane count. The 4-, 8- and 16-lane wrappers use explicit product alignment or result registers. Consequently, the multiplier latency, adder depth and wrapper registers must all be included when checking the timing of `out_valid` against the corresponding sum.

3.3 Accumulation and arithmetic width

For unsigned input elements,

$$P_{\max} = 255 \times 255 = 65{,}025, \tag{2}$$

$$S_{\max}(N) = N \times 65{,}025, \tag{3}$$

$$D_{\max}(1000) = 1000 \times 65{,}025 = 65{,}025{,}000. \tag{4}$$

A 16-bit product is sufficient because $P_{\max} < 2^{16}$. The maximum 16-lane beat sum is 1,040,400, which fits in 20 bits. A full 1000-element dot product requires 26 bits mathematically; the RTL uses a 32-bit accumulator. The current lane wrappers expose 18-, 18-, 19- and 20-bit beat sums for $N = 1, 4, 8, 16$, respectively.

The accumulator counts accepted beat sums and asserts `result_valid` at the end of a vector. For non-multiples of the lane count, correct arithmetic requires unused positions in the final beat to be zero or explicitly masked. The present top-level interface has no lane-valid mask.

4. Functional verification

The repository includes testbenches for the multiplier, lane wrappers, adder tree, accumulator and integrated vector datapath. Reference values are calculated using standard arithmetic expressions and compared with the observed outputs. The directed stimuli include zero operands, maximum unsigned values and selected mixed-bit patterns; additional tests use pseudo-random and back-to-back transactions.

Table 2. Existing verification targets in the repository.

Unit under test	Testbench	Principal checks
Wallace multiplier	`tb_wallace_unit.v`	Boundary, random and continuous inputs
Adder tree	`tb_adder_tree_var.v`	Summation, valid propagation and stress inputs
Accumulator	`tb_accumulator.v`	Beat counting and accumulated reference sums
Integrated datapath	`tb_vector_mac_top.v`	Multiple lane counts and vector-level arithmetic

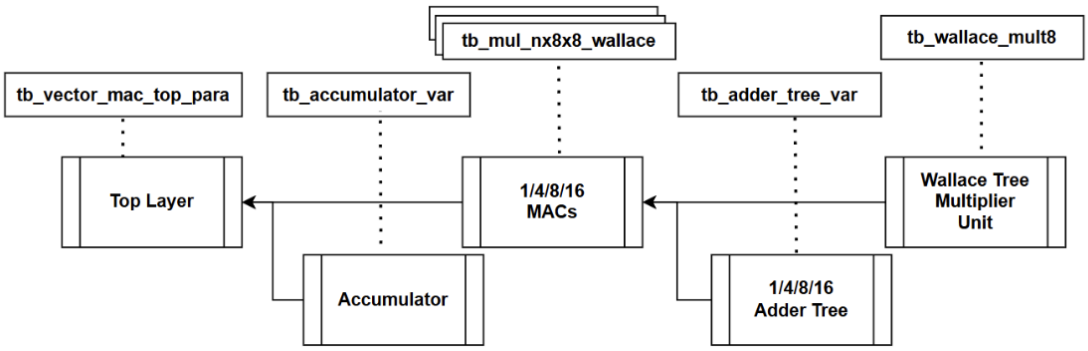


Figure 2. Module-level verification structure from the original design study. The labels identify testbench modules; actual source filenames are listed in Table 2.

These are existing verification assets, not evidence of a regression executed for this document. In particular, some benches print an error count but terminate with `$finish` regardless of the result. The integrated reference calculation also processes all 16 lane positions on the last beat of a 1000-element vector. A revised test should independently require only the final eight active positions to contribute. Additional checks should cover reset during traffic, missing results, and process-level failure on a mismatch or timeout.

5. FPGA implementation results

Table 3 reproduces the configurations reported in the original implementation study for an Artix-7 XC7A100T device. The reported 4.6–4.8 ns values have been relabeled as clock period rather than end-to-end pipeline latency, since they correspond approximately to $1/f_{\max}$. These are historical post-implementation summary values; the repository does not currently include the complete build scripts and reports needed to reproduce them. The 16-lane configuration is present in RTL but has no comparable result in the source table.

Table 3. Reported FPGA implementation summary (original design report).

Lanes	Period (ns)	$f_{\max}$ (MHz)	CLB eq.	Power (W)	Peak rate (MMAC/s)	Rate/CLB (MMAC/s/CLB)
1	4.736	211	39.60	0.010	211	5.33
4	4.603	217	133.85	0.023	868	6.49
8	4.780	209	285.85	0.042	1672	6.23

The tabulated peak multiply–accumulate issue rate follows

$$T_{\text{peak}} = N f_{\max}. \tag{5}$$

It assumes one valid set of N products per clock after pipeline fill. It does not include vector start-up and drain cycles, memory-transfer costs or downstream backpressure. The power entries are design estimates, not measurements from an instrumented FPGA board.

5.1 Area and throughput scaling

As shown in Fig. 3, the 4-lane configuration has the highest reported peak rate per equivalent CLB (6.49 MMAC/s/CLB). The 8-lane configuration has the highest total peak rate (1672 MMAC/s, or 1.672 GMAC/s) but a lower area-normalized value. Moving from 4 to 8 lanes increases peak rate by a factor of 1.93 and equivalent CLB area by a factor of 2.14. Under the reported mapping assumptions, four lanes therefore provide a better area-efficiency point among the three measured configurations.

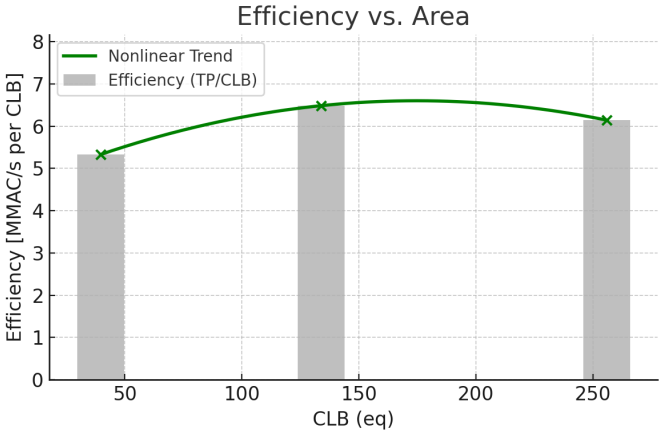


Figure 3. Original area-normalized throughput comparison for the 1-, 4- and 8-lane designs. The smooth trend line is a visualization between three reported data points, not a separate implementation measurement.

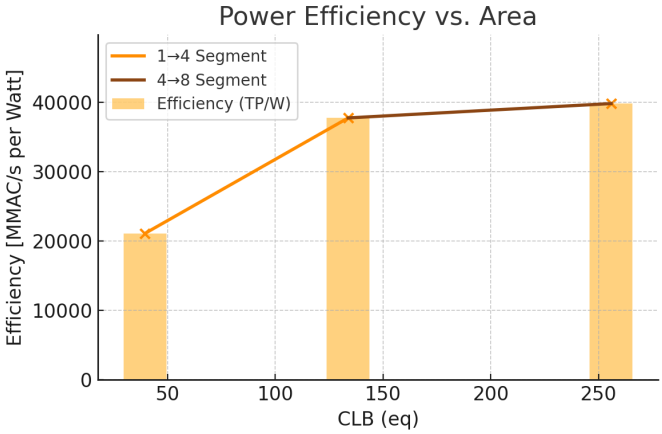


Figure 4. Original estimate of peak throughput per watt versus equivalent CLB area. The vertical scale is in MMAC/s/W; the corresponding values are approximately 21.1, 37.7 and 39.8 GMAC/s/W. Connecting lines are illustrative only.

Using the original estimated power values, the corresponding peak throughput per watt increases from approximately 21.1 GMAC/s/W (1 lane) to 37.7 (4 lanes) and 39.8 (8 lanes), as illustrated in Fig. 4. These comparisons represent implementation estimates and do not account for external memory activity or measured board power.

The reported frequencies lie between 209 and 217 MHz, but their comparison depends on valid timing constraints. In the public repository, `mul_clock.xdc` refers to names that do not match the integrated top-level ports and contains output false-path exceptions. Timing closure should not be reasserted without a corrected constraint set and a fresh implementation run.

6. Implementation boundaries and reproducibility

The present release should be interpreted as an unsigned, fixed-vector-length MAC datapath rather than a general-purpose streaming accelerator. The RTL does not demonstrate signed arithmetic, hardware sparse-data skipping, lane-level tail masking or ready/valid backpressure. The four lane configurations are selected at elaboration, and the wrappers use different register schedules.

For a reproducible implementation release, the first priorities are: (i) enforce fail-on-error simulation with a common reference model; (ii) verify all four lane configurations, including partial final beats and pipeline resets; (iii) align the XDC to the selected top-level interface; and (iv) archive the tool version, implementation Tcl, timing reports, utilization and power assumptions. These steps would make the existing area–throughput comparison independently auditable without changing the datapath's architectural intent.